Propaganda Detection

Student ID: 291745

Course: Advanced Natural Language Processing

1. Introduction

Identification of propaganda remains a vital computational linguistics task especially when solving online misinformation problems with narrative distortions. Automated systems which identify rhetorical propaganda enable disinformation control and public awareness of biased content particularly in political domains, social media and journalism. The task presents both significance and complexity to its execution because propaganda damages public trust while distorting democratic processes and distributes false information throughout worldwide media networks. However, the detection of propaganda proves difficult because certain words operate as propaganda in specific contexts though they remain neutral in other situations. The detection models need both to verify propaganda existence and recognize different rhetorical techniques which demands precise linguistic inspections of context. This report addresses propaganda detection in two stages. Task 1 assumes access to a known propaganda-containing span and focuses on classifying the associated technique. Task 2 presents a more realistic scenario, requiring the system to detect both the span and its technique directly from raw text. Each task is approached through three different methods. Task 1 leverages TF-IDF with Logistic Regression, BiLSTM, and fine-tuned BERT, while Task 2 employs heuristic span detection combined with logistic classification, BIO-tagging with BERT, and a QA-style span prediction model followed by technique classification. I evaluate the models using span-level accuracy, technique label accuracy, and full-match correctness, supported by detailed precision, recall, and F1-score reports for each class. The approaches use classical alongside neural networks to achieve maximum prediction accuracy along with clear interpretation value.

2. Dataset

The dataset stems from the Propaganda Techniques Corpus (PTC) which serves as a benchmark for identifying rhetorical propaganda in text according to Da San Martino et al. (2020). The dataset contains two files which include 2,560 sentences in the training set as well as 640 sentences in the validation set. The files present two columns containing label and sentence data in tab-separated format. The label represents one of nine fixed propaganda techniques which includes 'flag waving', 'appeal to fear prejudice', 'causal simplification', 'doubt', 'exaggeration, minimisation', 'loaded language', 'name calling, labeling', 'repetition' and 'not propaganda'. The label identifies the sentence as non-propaganda when no propaganda techniques are present. The sentence contains text annotations using <BOS> and <EOS> which specify the exact segment where the technique appears and serves as ground truth for span detection in Task 2 and context for classification in Task 1. Each sentence contains at most one labeled span and one associated technique, which simplifies the modeling process but does not reduce the linguistic complexity of the task.

3. Preprocessing and Exploratory Data Analysis (EDA)

This section involves outlining a preprocessing workflow while providing EDA findings which assisted future modeling decisions. The process consists of three parts including dataset preparation and model-oriented preprocessing and exploratory data analysis findings.

3.1 Data Cleaning and Feature Extraction

I first removed 10 duplicate entries from the training set. The dataset contained no missing values therefore no data imputation process was necessary. Each original sentence contained <BOS> and <EOS> tags marking the annotated propaganda span. Using regular expressions, I extracted the span and simultaneously removed the tags to obtain the cleaned sentence. The span was combined with its sentence to form combined_text, since context is often necessary for accurate technique classification. These features included the cleaned sentence (text), the span content (span), the associated technique (label), the span's normalized start index (relative_position), token counts for the span and sentence (span_length and text_length), and a combined sentence-span string (combined_text) used in classical ML models. These engineered features served as input to various models across both tasks. For instance, the logistic regression model processed numeric features along with TF-IDF features yet the deep learning models directly worked with the text and span fields.

label	*span*	*text*	*relative_position*	*span_length*	*text_length*	*combined_text*
not_propaganda	he	No, he will not be confirmed.	0.095238	1	6	he No, he will not be confirmed.
not_propaganda	won't make things any wors	This declassification effort won't	0.278846	11	14	won't make things any worse than
flag_waving	American people	The Obama administration misled	0.260563	2	21	American people The Obama adm
not_propaganda	and	It looks like we're capturing the d	0.295775	1	34	and It looks like we're capturing th
not_propaganda	Location: Westerville, Ohio	Location: Westerville, Ohio	0	3	3	Location: Westerville, Ohio Locati

Figure 1. Structured data includes spans and sentence features resulting after pre-processing

3.2 Model-Specific Preprocessing

The text tokenization and label encoding methods followed individual specifications for each model implementation. For classical model logistic regression, I used scikit-learn's TfidfVectorizer for creating sparse representations alongside engineered numerical features that included relative position, span length and sentence length. These numeric features were standardized using StandardScaler to ensure compatibility with linear classifiers. The Keras's Tokenizer function converted words to integer sequences when using LSTM-based models but BERT utilized HuggingFace's Transformers library for tokenization. All input data received the same uniform padding and truncation treatments to keep all entries within consistent lengths.

The technique labels received integer values through LabelEncoder to maintain uniform mappings between different models. All rhetorical technique labels received numerical representations through LabelEncoder which also served as a consistent framework for comparing model performance across all models. The approach for Task 2 differed because the span length was undisclosed. I designed BIO-tagged sequences for token-level classification and developed QA-style inputs for span prediction through start and end logits. Each token received its assigned label as B-X, I-X or O through the tokenization process in both tasks.

3.3 Exploratory Data Analysis (EDA)

I studied the dataset to understand the distribution patterns of tokens, span characteristics and class imbalance. The data shows an extreme class distribution because more than half of instances fall into the not_propaganda category. Next, I analysed sentence length across different classes. Most sentences are distributed between 25 and 35 words yet display varying average lengths according to technical specifications. Sentences labeled as 'causal_oversimplification' and 'name_calling, labeling' contain more words which demands models to analyze larger contextual information. Finally, I analysed the length of propaganda spans. Most spans contain between 3 and 7 words, with 'repetition' and 'loaded_language' having the shortest spans. The high frequency of short spans found in longer sentences makes token-level classifiers and QA style models suitable for detection work. Insights from EDA directly influenced selection of models. Fixed-size sliding windows were used because of short span lengths, while BIO tagging was applied for token-level sequence models. Macro-averaged evaluation metrics were chosen because of severe class imbalance observed across labels.

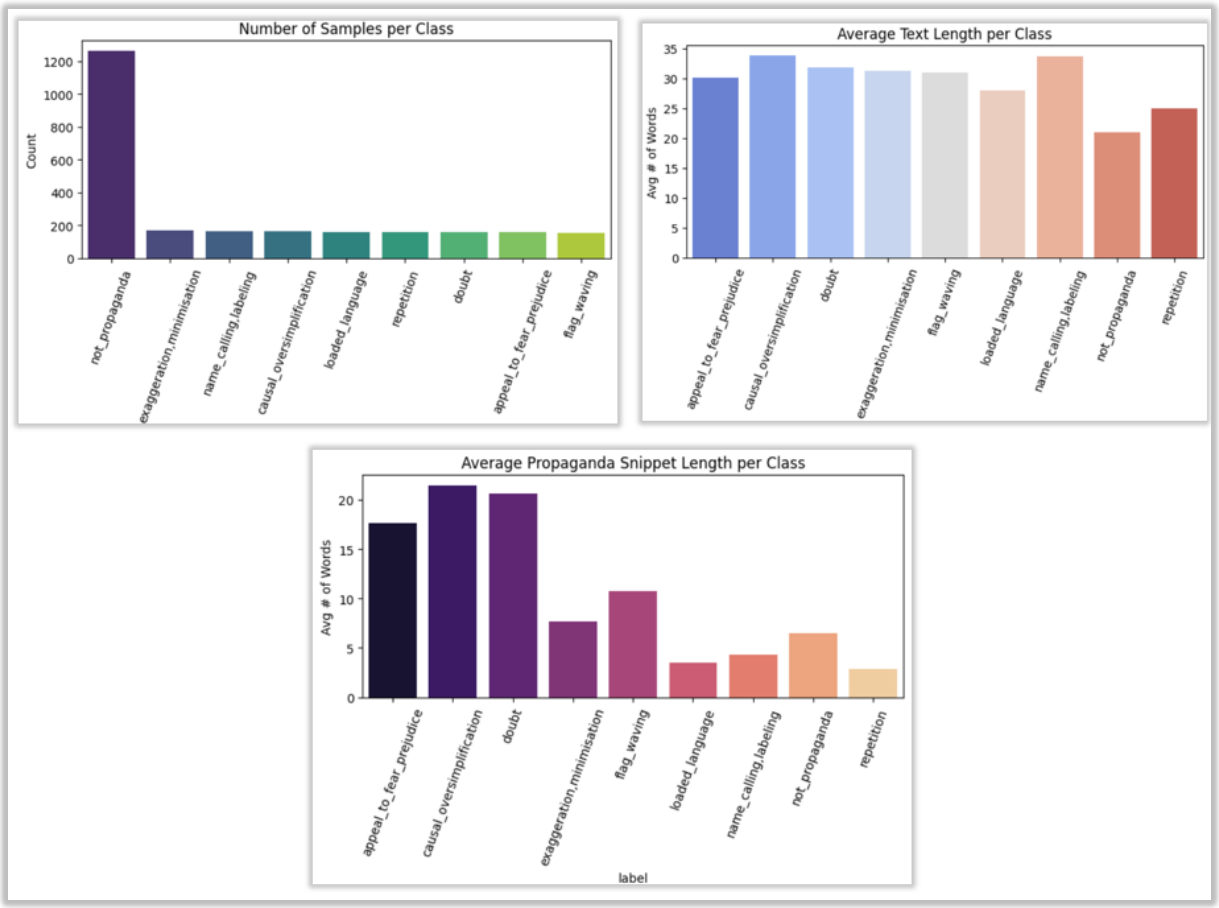

Figure 2. Class Distribution and Text Characteristics in the Propaganda Dataset

4. Methodology

The following part outlines the methods and models that remedy the core propaganda detection issues including both the recognition of propaganda techniques in defined spans and simultaneous detection of spans and techniques from raw texts. The system uses both classical classifiers with deep learning models alongside transformer-based architectures to fulfill its functionality. Models were developed to balance understanding, operational speed, and result effectiveness.

4.1 Task 1: Span and Technique Detection within Sentences

Task 1 assumes that the propaganda-containing span is already provided. The objective is to classify the technique used. I treated this as a multi-class classification problem. The approaches include propaganda span merged with its full sentence (combined_text) instead of using the span alone, because propagandistic techniques often need extensive context to reveal their rhetorical purpose. The isolated short phrases "our values" or "the threat" present as neutral expressions but become manipulative within some specific contexts. The absence of contextual information makes it hard for models to differentiate among neutral language and propaganda linguistic patterns. The chosen methodology applies concepts from pragmatics in linguistic theory since discourse shapes meaning while following previous work on propaganda detection which focused on analyzing sentence structures (Da San Martino et al., 2020). Therefore, the accuracy of technique classification increases when context information is added.

4.1.1 Approach 1: TF-IDF with Logistic Regression

The initial method uses engineered features with classical ML techniques. According to Sebastiani (2002) using the combined text, I generated TF-IDF features from unigrams and bigrams while restricting the terms to 5,000 to minimize dimensionality without losing key lexical signals. I engineered numerical features to measure span position relative to sentence length and token counts, which were standardized using StandardScaler. These features were combined with the TF-IDF vectors and passed to a multinomial logistic regression classifier. This approach was selected because it provides lightweight interpretation while working well for cases where predictive signals rely on features. Logistic regression has proven its effectiveness as a benchmark model in propaganda detection across various tasks such as SemEval and NLP4IF shared tasks (Gorrell et al., 2019; Da San Martino et al., 2019). I improved performance via grid search on the C regularization and class_weight parameters using 3-fold cross-validation and macro-averaged F1 score as the criterion.

4.1.2 Approach 2: BiLSTM with Trainable Embeddings

The second methodology applied Bidirectional Long Short-Term Memory (BiLSTM) networks for modeling sequential dependencies between sentences within the specified text span. The combined text (span with its sentence) tokenized using Keras's Tokenizer and padded to a fixed length for consistent input representation. Output labels were integer-encoded using LabelEncoder, then one-hot encoded to meet the requirements of softmax output layer. The trainable embedding layer acquired semantic token representations from the input after tokenization. This method was chosen because it can process information in both directions which proves essential for detecting rhetorical patterns dependent on sentence position. The LSTM output passed through a dense layer with dropout before generating a multi-class softmax prediction. Kim (2014) initiated text classification deep learning as well as Graves and Schmidhuber (2005) introduced sequence tagging which represents the foundation of this approach. The method succeeded in better representation learning yet it demonstrated was more sensitive to training setup and label imbalance.

4.1.3 Approach 3: Fine-Tuned BERT for Technique Classification

In this approach, I fine-tuned a pretrained BERT model for multi-class sequence classification. The Bidirectional Encoder Representations from Transformers model known as BERT was introduced by Devlin et al. (2019) to provide excellent results on various sentence-level Natural Language Processing tasks. I used the bert-base-uncased variant from the HuggingFace Transformers library, training it to classify propaganda techniques based on combined span and sentence inputs. BertTokenizer processed the data while truncating and padding sequences to 128 maximum tokens. The LabelEncoder method applied consistent label mappings between training and evaluation phases. A custom PyTorch Dataset object contained the encoded inputs and labels before DataLoader with batching and GPU acceleration loaded them. I selected BertForSequenceClassification as the model architecture, which appends a softmax classification head on top of the [CLS] token. The training adopted the AdamW optimizer. I chose BERT because this model delivers context-rich vector representations that detect fine rhetorical elements present in both spans and sentences. The proposed method applies recent transformer fine-tuning strategies for downstream tasks and follows detection practices established in recent propaganda benchmarks (Da San Martino et al., 2020; Rogers et al., 2020). All transformer-based models were trained using GPU acceleration on Google Colab.

4.2 Task 2: Propaganda Technique Classification (Given Span)

Task 2 involved detecting both the propaganda span and its technique from raw sentences. The first approach applied BERT with BIO tagging to jointly predict token-level spans and their corresponding techniques. The pipeline in the second approach applied two-stage operation: it first extracted spans with a heuristic method before moving onto a logistic regression classifier. The third adopted a QA-style span predictor combined with a fine-tuned BERT classifier. Only the first method predicted spans and techniques jointly, whereas the other two approaches separated the tasks to better isolate errors and reuse components efficiently. This separation was particularly useful in managing span prediction noise and allowed reusing the Task 1 classifier for improved efficiency for the second approach.

4.2.1 Approach 1: BERT-Based Token Classification Using BIO Tagging

In this approach, I treated the problem as a sequence labeling task. I used the standard BIO tagging format, assigning B-technique for span beginnings, I-technique for continuation, and O for tokens outside any span. Although a cleaned dataset with separate span and sentence columns had been created, I used the original tagged_in_context field to accurately align tokenized spans with sentence context using the <BOS> and <EOS> markers. The alignment was performed using BertTokenizer, and the resulting BIO-labeled token sequences were passed to BertForTokenClassification for training.

Furthermore, the bert-base-uncased model underwent training with token-level cross-entropy loss which used AdamW optimization through three training epochs. After training, I simplified the predicted BIO tags by removing prefix to focus on technique assigned to each span. The BIO tagging method in this work adopts the standard from named entity recognition (Tjong Kim Sang & De Meulder, 2003) which has shown successful results in propaganda detection tasks (Da San Martino et al., 2020; Rogers et al., 2020). This method executes label prediction as well as span extraction through a unified algorithm, while showing token alignment problems.

4.2.2 Approach 2: Heuristic Span Selection Followed by Logistic Regression

The proposed method addresses Task 2 through a two-stage process: span prediction using heuristics followed by logistic regression for technique classification. I identified spans using a sliding window that selects the n-gram (window size 3-5) with the highest TF-IDF average score. The method aimed to create a lightweight substitute for full sequence tagging or span-based transformers preserves the essential ability to detect span boundaries from raw text. The method uses TF-IDF weighting to detect distinctive propaganda phrases, a principle that established by Mihalcea & Tarau (2004). The predicted span is then combined with the original sentence and passed to the logistic regression model from Task 1, using the existing TF-IDF vectorizer and numeric features. The evaluation process remains consistent because re-training is not necessary. The evaluation of span accuracy uses Jaccard overlap, reporting span match rates (>30% overlap), label accuracy, and full-match accuracy (both span and technique match). This method provides limited precision but remains fast and easy to understand so it functions as a useful practical baseline.

4.2.3 Approach 3: Span Detection via QA-Style BERT with BERT Classifier

This method employed a question answering framework for span detection followed by BERT-based classification for technique prediction. This method treats the propaganda span as the answer to an implicit question: “What part of this sentence is propagandistic?”.

Training data was prepared by using the original sentence as context and annotating the character-level start and end positions of the gold span. BertTokenizerFast processed the examples through tokenization which made it possible to link token positions with original character positions. These aligned inputs trained a BertForQuestionAnswering model to predict the start and end token positions of the propaganda span. The training process utilized five epochs and AdamW optimizer. The inference process involved converting predicted token into textual spans using offset mappings. Span match accuracy was calculated using both exact match and Jaccard overlap. Once spans were predicted, they were combined with the original sentence and passed to the BERT classifier trained in Task 1 for technique assignment. The final evaluation included span accuracy, technique accuracy, and full match accuracy (both span and label correct). The method builds on extractive QA systems from Rajpurkar et al., (2016) and span-based model adaptations from Da San Martino et al. (2020) as its foundation. The span alignment procedure remained interpretable yet its performance deteriorated during comprehensive system execution.

5. Hyperparameter Settings

Model performance and training stability depends on hyperparameters, which also affect the generalization level. I used the standard values from existing literature and grid search optimization of selected parameters across models.

The following section outlines the fundamental hyperparameters that were either fixed value or tuned for each approach.

5.1 Task 1: Propaganda Technique Classification

For approach 1, TfidfVectorizer extracted unigram and bigram features through ngram_range=(1, 2) while keeping the terms to 5,000. Numerical features were standardized using StandardScaler. The logistic regression model was trained with max_iter = 1000 to ensure convergence. I used GridSearchCV to tune regularization strength (C ∈ {0.01, 0.1, 1, 10}) and class_weight (None or 'balanced'), selecting the best configuration based on macro-averaged F1 score using 3-fold cross-validation. For approach 2, I used Keras's Tokenizer with a vocabulary of 10,000 words and padded input sequences to a fixed length of 100. The embedding layer had an output dimension of 200, followed by a BiLSTM with 128 hidden units and two dropout layers (rates: 0.3 and 0.2). The model was trained using Adam, categorical cross-entropy loss, batch size of 32, and 20 epochs. For approach 3, I used bert-base-uncased with a sequence length of 128 tokens. A model training process used AdamW optimization with 2e-5 learning rate and 16 batch size for five epochs. The BertTokenizer processed all inputs by applying tokenization methods with padding and truncation.

5.2 Task 2: Propaganda Technique Classification

For approach 1, BIO labels were produced through BertTokenizer before BertTokenizerFast performed alignment operations to maintain correct word-to-label correspondence. A model training process used AdamW optimization with 2e-5 learning rate and 16 batch size for five epochs. For approach 2, I reused the logistic regression model and feature pipeline from Task 1. For span detection, I tested sliding window sizes of 3, 4, and 5 tokens and selected 5 based on Jaccard overlap performance. For approach 3, I trained BertForQuestionAnswering for span detection. The optimization process used AdamW with 3e-5 learning rate and 16-batch size training the model for five epochs. The token predictions were linked back to character-level spans using offset mappings from BertTokenizerFast. For all transformer-based models, I used bert-base-uncased with a sequence length of 128 tokens.

6. Results and Evaluation

6.1 Task 1: Technique Classification

6.1.1 Approach 1: TF-IDF + Logistic Regression

The first logistic regression model obtained 57% accuracy when validated against the test data. The model demonstrated high F1-score of 0.73 on the 'not_propaganda' class but showed poor performance on the minority classes 'loaded_language', 'repetition' and 'exaggeration, minimisation' leading to a low macro-averaged F1-score of 0.27 due to class imbalance. The majority of errors appeared as misidentifications into 'not_propaganda' category.

The combination of C = 10 and class_weight = 'balanced' in GridSearchCV enabled an improvement of macro-averaged F1-score to 0.41 while accuracy stayed unchanged. Classes such as 'appeal_to_fear_prejudice', 'causal_oversimplification', and 'doubt' increased precision and recall quantities due to the essential role of reweighting in unbalanced datasets. The class-level balance improvements in the model did not result in increased overall accuracy because feature expressiveness together with model capacity limited the results to 57%. The obtained findings demonstrate that weight rebalancing proves useful with imbalanced data sets at the same time it shows that TF-IDF features perform inadequately for detecting nuanced rhetorical indicators.

label	*precision*	*recall*	*f1-score*	*support*
appeal_to_fear_prejudice	0.39	0.4	0.39	43
causal_oversimplification	0.41	0.43	0.42	35
doubt	0.43	0.47	0.44	43
exaggeration, minimisation	0.28	0.33	0.3	30
flag_waving	0.61	0.6	0.61	45
loaded_language	0.17	0.13	0.15	39
name_calling, labeling	0.28	0.24	0.25	34
not_propaganda	0.74	0.76	0.75	331
repetition	0.35	0.33	0.34	40
accuracy			0.57	640
macro avg	0.41	0.41	0.41	640
weighted avg	0.57	0.57	0.57	640

Figure 3. Classification Report (After Tuning)

6.1.2 Approach 2: BiLSTM with Trainable Embeddings

The BiLSTM achieved 55% validation set accuracy with a 0.35 macro-averaged F1-score as shown in figure 4. While slightly lower than the logistic regression baseline, it showed moderate gains in minority classes such as 'exaggeration, minimisation' and 'flag_waving'. Different combinations of embedding sizes and hidden units produced no significant change in accuracy which stayed between 45% and 55%. The model exhibited capabilities in pattern sequence modeling but its strong responsiveness to data class proportions and dataset size reduced the ability to optimize generalization.

label	*precision*	*recall*	*f1-score*	*support*
appeal_to_fear_prejudice	0.3	0.16	0.21	43
causal_oversimplification	0.38	0.34	0.36	35
doubt	0.26	0.26	0.26	43
exaggeration, minimisation	0.3	0.43	0.36	30
flag_waving	0.54	0.49	0.51	45
loaded_language	0.2	0.15	0.17	39
name_calling, labeling	0.33	0.12	0.17	34
not_propaganda	0.7	0.8	0.75	331
repetition	0.34	0.3	0.32	40
accuracy			0.55	640
macro avg	0.37	0.34	0.35	640
weighted avg	0.52	0.55	0.53	640

Figure 4. Classification Report for BiLSTM Model

6.1.3 Approach 3: Fine-Tuned BERT Classifier

The fine-tuned BERT classifier reached validation accuracy of 72% and macro-averaged F1-score of 0.59 which surpassed the performance of other approaches (figure 5). It produced outstanding performance across all classes by reaching F1-scores of 0.73 for 'name_calling, labeling', 0.67 for 'flag_waving' and 0.65 for 'exaggeration, minimisation'. The model also maintained excellent precision and recall values on the not_propaganda category demonstrating the advantage of contextual embeddings work well for rhetorical classification purposes. However, class imbalance acted as a limiting factor for performance on the detection of rare techniques.

label	*precision*	*recall*	*f1-score*	*support*
appeal_to_fear_prejudice	0.46	0.51	0.48	43
causal_oversimplification	0.46	0.6	0.52	35
doubt	0.57	0.3	0.39	43
exaggeration, minimisation	0.72	0.6	0.65	30
flag_waving	0.66	0.69	0.67	45
loaded_language	0.66	0.49	0.56	39
name_calling, labeling	0.75	0.71	0.73	34
not_propaganda	0.92	0.88	0.9	331
repetition	0.32	0.6	0.42	40
accuracy			0.72	640
macro avg	0.61	0.6	0.59	640
weighted avg	0.75	0.72	0.73	640

Figure 5. Classification Report for BERT Model

6.2 Task 2: Span and Technique Detection Results

6.2.1 Approach 1: BERT-Based Token Classification Using BIO Tagging

The BIO tagging implementation of BERT the model demonstrated 63% token-level accuracy across the validation dataset with a macro-averaged F1-score of 0.18 (figure 6). Most tokens were correctly labeled as O (non-propaganda), with an F1-score of 0.78, while propaganda techniques had F1-scores below 0.35. The span detection method succeeded partially however loss reweighting or better sequence models may provide additional gains.

label	*precision*	*recall*	*f1-score*	*support*
O	0.68	0.92	0.78	14764
appeal_to_fear_prejudice	0.27	0.46	0.34	1031
causal_oversimplification	0.68	0.08	0.14	1071
doubt	0	0	0	1321
exaggeration, minimisation	0.29	0.18	0.22	319
flag_waving	0.47	0.12	0.19	656
loaded_language	0	0	0	247
name_calling, labeling	0	0	0	283
not_propaganda	0.28	0.07	0.11	2965
repetition	0	0	0	160
accuracy			0.63	22817
macro avg	0.27	0.18	0.18	22817
weighted avg	0.54	0.63	0.55	22817

Figure 6. Classification Report for BERT BIO Tagging

6.2.2 Approach 2: Heuristic Span Selection Followed by Logistic Regression

The heuristic method used for span detection showed worse performance than learned models did. The better span match accuracy of 29.38% was achieved using sliding windows with a size of 5 applied to TF-IDF scores. The label accuracy of the reused logistic regression model from Task 1 reached 49.84% however complete match precision amounted to 16.41% due to independent errors in span and technique detection. The procedure performs fast and easily understandable yet it fails to capture the detailed contextual information required for precise identification of spans.

6.2.3 Approach 3: Question Answering-Based Span Extraction with BERT Classification

The QA-style BERT model achieved 44.84% accuracy in detecting partial span matches when the Jaccard threshold greater than 30%. The evaluation based on partial span matches proved more productive than exact span matches which occurred only in 6.56% of cases. The BERT sequence classifier achieved 50.16% accuracy for technique classification after performing span extraction. Full match accuracy (both the span and technique are correct) was 24.38% only. The Figure 7 displays successful examples of span and label matches. The method yielded better span identification results than heuristic methods but boundary detection mistakes from this stage affected the classification outcomes.

True Span	*Predicted Span*	*Jaccard Overlap*	*Span Match*	*Predicted Technique*	*True Technique*	*Technique Match*	*Full Match*
gets Earl Warren and Sen. Richard Russel to jo…	telling them that the assassination could lead…	0.52	TRUE	appeal_to_fear_prejudice	appeal_to_fear_prejudice	TRUE	TRUE
our	our leaders	0.4	TRUE	flag_waving	flag_waving	TRUE	TRUE
we are immensely grateful for all the massive ...	immensely grateful for all the massive support…	0.48	TRUE	exaggeration,minimisation	exaggeration,minimisation	TRUE	TRUE
Why are your bishops silent on a threat that 1...	Why are your bishops silent on a threat that 1…	0.93	TRUE	doubt	doubt	TRUE	TRUE
Bo Mueller is totally conflicted, and his 17 ...	his 17 Angry Democrats that are doing this dir…	0.68	TRUE	flag_waving	flag_waving	TRUE	TRUE
OUR SOUTHERN BORDER!	OUR SOUTHERN BORDER!	0.5	TRUE	flag_waving	flag_waving	TRUE	TRUE
Latino Voters Show Trump What It Means to Be A...	Trump What It Means to Be American	0.7	TRUE	flag_waving	flag_waving	TRUE	TRUE
Instead of reaching out to the greatest demogr...	They send missionaries to foreign lands to do…	0.54	TRUE	flag_waving	flag_waving	TRUE	TRUE
The enemy is not only at the gates but has now...	The enemy is not only at the gates but has now…	1	TRUE	appeal_to_fear_prejudice	appeal_to_fear_prejudice	TRUE	TRUE

Figure 7. Example Outputs Showing Span and Technique Full Matches (QA BERT + BERT Classifier)

6.3 Overall Comparison of Approaches

The below figure shows the overall performance comparison across all approaches for task 1 and task 2

Task	Approach	Span Match Accuracy	Technique Accuracy	Full Match Accuracy	Macro F1-Score	Overall Comment
1	TF-IDF + Logistic Regression	-	57%	-	0.41	Strong baseline after class balancing
	BiLSTM with Trainable Embeddings	-	55%	-	0.35	Sequence modeling but sensitive to imbalance
	Fine-Tuned BERT Classifier	-	72%	-	0.59	Best overall for Task 1
2	BERT BIO Tagging	Token level: 63% (mostly O class)		-	0.18	High background accuracy, low span technique recall
	Heuristic + Logistic Regression	29.38% (window=5)	49.84%	16.41%	-	Fast but inaccurate span prediction
	QA BERT Span + BERT Technique Classifier	44.84% (>30% Jaccard)	50.16%	24.38%	-	Best for span detection in Task 2

Figure 8. Overall Performance Comparison Across All Approaches for Task 1 and Task 2

7. Conclusion

This project conducted multiple methods to detect propaganda by examining technique classification (Task 1) and combined span and technique detection (Task 2). In Task 1, the deep learning model utilizing fine-tuned BERT outperformed other detection models by relying on contextual embeddings to reach better macro F1-scores. The deep learning model that used fine-tuned BERT achieved superior performance in Task 1 through contextual embedding detection which resulted in better macro F1-scores. In Task 2, span detection was the primary bottleneck. Both heuristic approaches failed to work effectively while the combination of BIO tagging and BERT proved better at detecting tokens, yet it often misidentified background elements. The QA-style BERT model excelled at partial span identification but it did not succeed in stopping boundary classification errors. I first thought for task 2, a two-stage pipeline would perform better than a single joint model as it divided span detection from technique classification but the approach 1 produced superior results because it minimized errors and learned token-label relationships directly. Furthermore, context-specific system models delivered superior outcomes than feature-based models yet precision of span detection and unbalanced classes continue to be major obstacles for reaching high full-match accuracy. Future end-to-end propaganda detection systems need to overcome these current problems for successful development.

8. Further Work

Future research should concentrate on enhancing span detection accuracy as this stands as the main performance barrier for complete systems. Research should explore span-based transformer models to jointly predict span boundaries along with techniques since this approach helps reduce errors that spread across processing stages. The use of data augmentation methods through adversarial examples and paraphrasing would help balance classes and enhance generalization capability in systems. Additionally, external knowledge sources of rhetorical strategies would help improve model interpretability and make classifications more robust when incorporated into the system. Research should explore the development of lightweight models because they would provide the ability to detect propaganda in real time during practical usage. Industrial progress should consider the pairing of RoBERTa models with CRF decoding components based on SemEval-2020 findings by Da San Martino et al. (2020) that report superior span identification results.

References

Da San Martino, G., Yu, S., Barrón-Cedeño, A., & Nakov, P. (2020). PRTA: A system to support the analysis of propaganda techniques in the news. *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics: System Demonstrations*, 287–293.

Devlin, J., Chang, M.-W., Lee, K., & Toutanova, K. (2019). *BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding*. Proceedings of NAACL-HLT.

Gorrell, G., Kochkina, E., Kontar, S., Augenstein, I., & Bontcheva, K. (2019). SemEval-2019 Task 4: Hyperpartisan news detection. *Proceedings of the 13th International Workshop on Semantic Evaluation (SemEval-2019)*, 829–839.

Graves, A., & Schmidhuber, J. (2005). Framewise phoneme classification with bidirectional LSTM and other neural network architectures. *Neural Networks*, 18(5–6), 602–610.

Kim, Y. (2014). Convolutional neural networks for sentence classification. *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 1746–1751.

Mihalcea, R., & Tarau, P. (2004). TextRank: Bringing order into texts. *Proceedings of the 2004 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 404–411.

Rajpurkar, P., Zhang, J., Lopyrev, K., & Liang, P. (2016). SQuAD: 100,000+ questions for machine comprehension of text. *Proceedings of the 2016 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2383–2392.

Rogers, A., Pereira, A., & Augenstein, I. (2020). What's in your head? A toolkit for propaganda detection. *Proceedings of the ACL 2020 Workshop on NLP for COVID-19 (NLP-COVID)*, 77–84.

Tjong Kim Sang, E. F., & De Meulder, F. (2003). Introduction to the CoNLL-2003 shared task: Language-independent named entity recognition. *Proceedings of the Seventh Conference on Natural Language Learning at HLT-NAACL 2003*, 142–147.